МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение высшего образования «Санкт-Петербургский политехнический университет Петра Великого»

Институт компьютерных наук и кибербезопасности

Высшая школа технологий и искусственного интеллекта
Направление: 02.03.01 Математика и компьютерные науки

Программирование микроконтроллеров
Отчет о выполнении лабораторной работы №1
Тема лабораторной работы: Создание нового проекта в среде Keil µVision5

Выполнил:
Студент, группы
5130201/40001 _____ Ови Мд Шамин Ясир

Проверил:
Преподаватель,
_____ Вербова Н. М.

«____»_____ 20__ г.

Санкт-Петербург, 2026

Содержание

1 Цель работы 3

2 Постановка задачи 3

3 Особенности реализации 3

4 Код программы 7

5 Выводы 9

1 Цель работы

Ознакомиться с основными приёмами работы в среде разработки Keil μVision5, а также получить навыки создания нового проекта для микроконтроллера STM32F200 и разработки программы управления светодиодом с использованием регистров ввода-вывода.

2 Постановка задачи

Создать новый проект в среде Keil μVision5 для микроконтроллера STM32F200 и разработать программу, обеспечивающую периодическое включение и выключение светодиода с использованием регистров ввода-вывода.

3 Особенности реализации

Реализация программы основана на непосредственной работе с регистрами микроконтроллера STM32F200, без использования высокоуровневых библиотек.

Согласно принципиальной схеме оценочной платы (рис. 1), светодиоды подключены к выводам микроконтроллера через буферную микросхему 74LVC244T. Управление светодиодами осуществляется посредством подачи логического уровня на соответствующие выводы портов (в частности, PG7).

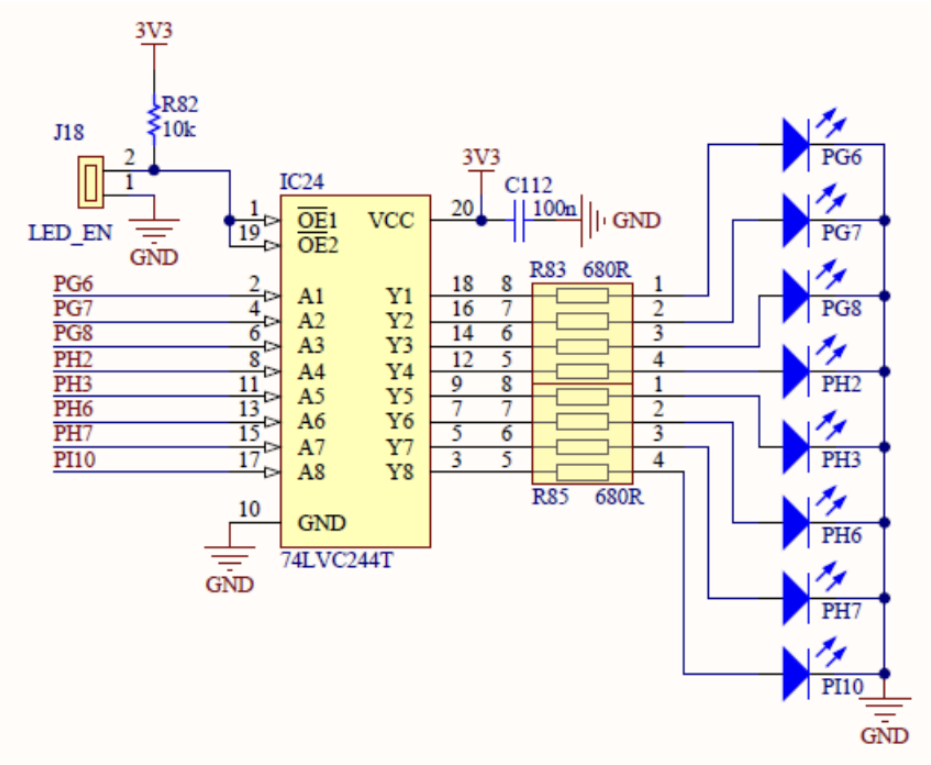

Рис. 1: Схема подключения светодиодов

Перед использованием порта необходимо включить тактирование соответствующего периферийного блока. По умолчанию после подачи питания большинство периферийных модулей отключены с целью снижения энергопотребления. Включение выполняется через регистр RCC_AHB1ENR, путем установки бита GPIOGEN (бит 6) в состояние логической единицы.

5.3.10 RCC AHB1 peripheral clock register (RCC_AHB1ENR)

Address offset: 0x30

Reset value: 0x0000 0000

Access: no wait state, word, half-word and byte access.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reser-ved	OTGHS ULPIEN	OTGHSEN	ETHMACPTPEN	ETHMACRXEN	ETHMACTXEN	ETHMACEN	Reserved		DMA2EN	DMA1EN	Reserved		BKPSRAMEN	Reserved	
	rw	rw	rw	rw	rw	rw			rw	rw			rw		

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved			CRCEN	Reserved			GPIOIEN	GPIOHEN	GPIOGEN	GPIOFEN	GPIOEEN	GPIODEN	GPIOCEN	GPIOBEN	GPIOAEN
			rw				rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit 6 **GPIOGEN:** IO port G clock enable
Set and cleared by software.
0: IO port G clock disabled
1: IO port G clock enabled

Рис. 2: Регистр RCC_AHB1ENR

Адрес регистра RCC_AHB1ENR определяется на основе карты па-

мяти микроконтроллера. Базовый адрес блока RCC равен 0x40023800, а смещение регистра составляет 0x30, следовательно, итоговый адрес регистра равен 0x40023830.

Table 1. STM32F20x and STM32F21x register boundary addresses

Boundary address	Peripheral	Bus	Register map
0xA000 0000 - 0xA000 0FFF	FSMC control register	AHB3	*Section 31.6.9: FSMC register map on page 1300*
0x5006 0800 - 0X5006 0BFF	RNG	AHB2	*Section 20.4.4: RNG register map on page 537*
0x5006 0400 - 0X5006 07FF	HASH		*Section 21.4.8: HASH register map on page 557*
0x5006 0000 - 0X5006 03FF	CRYP		*Section 19.6.11: CRYP register map on page 531*
0x5005 0000 - 0X5005 03FF	DCMI		*Section 12.8.12: DCMI register map on page 299*
0x5000 0000 - 0X5003 FFFF	USB OTG FS		*Section 29.16.6: OTG_FS register map on page 1028*
0x4004 0000 - 0x4007 FFFF	USB OTG HS	AHB1	*Section 30.12.6: OTG_HS register map on page 1172*
0x4002 9000 - 0x4002 93FF	ETHERNET MAC		*Section 28.8.5: Ethernet register maps on page 941*
0x4002 8C00 - 0x4002 8FFF			
0x4002 8800 - 0x4002 8BFF			
0x4002 8400 - 0x4002 87FF			
0x4002 8000 - 0x4002 83FF			
0x4002 6400 - 0x4002 67FF	DMA2		*Section 9.5.11: DMA register map on page 208*
0x4002 6000 - 0x4002 63FF	DMA1		
0x4002 4000 - 0x4002 4FFF	BKPSRAM		
0x4002 3C00 - 0x4002 3FFF	Flash interface register		See Flash programming manual
0x4002 3800 - 0x4002 3BFF	RCC		*Section 5.3.24: RCC register map on page 135*
0x4002 3000 - 0x4002 33FF	CRC		*Section 3.4.4: CRC register map on page 61*
0x4002 2000 - 0x4002 23FF	GPIOI		*Section 6.4.11: GPIO register map on page 156*
0x4002 1C00 - 0x4002 1FFF	GPIOH		
0x4002 1800 - 0x4002 1BFF	GPIOG		
0x4002 1400 - 0x4002 17FF	GPIOF		
0x4002 1000 - 0x4002 13FF	GPIOE		
0X4002 0C00 - 0x4002 0FFF	GPIOD		
0x4002 0800 - 0x4002 0BFF	GPIOC		
0x4002 0400 - 0x4002 07FF	GPIOB		
0x4002 0000 - 0x4002 03FF	GPIOA		

Рис. 3: Карта распределения адресов микроконтроллера

После включения тактирования необходимо настроить режим работы выводов. Для этого используются регистры GPIOx_MODER, где каждый вывод настраивается двумя битами.

Для перевода вывода PG7 в режим выхода необходимо записать значение 01 в биты 15-14 регистра GPIOG_MODER:

```
*(unsigned long*)(0x40021800) =
(*(unsigned long*)(0x40021800) & (~0x00008000)) | (0x00004000
  );
```

Аналогично выполняется настройка других портов. Например, для PH2:

```
*(unsigned long*)(0x40021C00) =
(*(unsigned long*)(0x40021C00) & (~0x00000020)) | (0x00000010
   );
```

Для PI10:

```
*(unsigned long*)(0x40022000) =
(*(unsigned long*)(0x40022000) & (~0x00200000)) | (0x00100000
   );
```

Перед настройкой портов также необходимо включить их тактирование:

```
*(unsigned long*)(0x40023830) |= 0x40 | 0x80 | 0x100;
```

После настройки режимов управление состоянием выводов осуществляется через регистр GPIOx_ODR. Запись логической единицы приводит к включению светодиода, а запись нуля — к его выключению.

В основном цикле программы реализуется последовательное включение и выключение светодиодов:

```
while(1)
{
    // PG6
    *(unsigned long*)(0x40021814) |= 0x40;
    for(int j=0; j<2000000; j++){}
    *(unsigned long*)(0x40021814) &= ~0x40;
    for(int j=0; j<2000000; j++){}

    // PG7
    *(unsigned long*)(0x40021814) |= 0x80;
    for(int j=0; j<2000000; j++){}
    *(unsigned long*)(0x40021814) &= ~0x80;
    for(int j=0; j<2000000; j++){}
}
```

Дополнительно параметры работы выводов (тип, скорость, подтяжка) могут настраиваться через регистры GPIOx_OTYPER, GPIOx_OSPEEDR и GPIOx_PUPDR, однако в данной работе используются значения по умолчанию.

Таким образом, реализация программы включает следующие этапы:

- включение тактирования портов;
- настройка выводов в режим выхода;
- управление состоянием выводов через регистр GPIOx_ODR;
- организация временных задержек программным способом.

4 Код программы

```
int main() {

    int i; // counter for get ready delay
    unsigned long int j; // counter for blinky delay

    i = 0;
    j = 0;

    // Enable clock for GPIOG, GPIOH, GPIOI
    *(unsigned long*)(0x40023830) |= 0x40 | 0x80 | 0x100;

    for(i = 0; i < 4; i++) {}

    // PG6
    *(unsigned long*)(0x40021800) =
    (*(unsigned long*)(0x40021800) & (~0x00002000)) | (0
       x00001000);

    // PG7
    *(unsigned long*)(0x40021800) =
    (*(unsigned long*)(0x40021800) & (~0x00008000)) | (0
       x00004000);

    // PG8
    *(unsigned long*)(0x40021800) =
    (*(unsigned long*)(0x40021800) & (~0x00020000)) | (0
       x00010000);

    // PH2
    *(unsigned long*)(0x40021C00) =
    (*(unsigned long*)(0x40021C00) & (~0x00000020)) | (0
       x00000010);

    // PH3
    *(unsigned long*)(0x40021C00) =
    (*(unsigned long*)(0x40021C00) & (~0x00000080)) | (0
       x00000040);

    // PH6
    *(unsigned long*)(0x40021C00) =
    (*(unsigned long*)(0x40021C00) & (~0x00002000)) | (0
       x00001000);

    // PH7
    *(unsigned long*)(0x40021C00) =
```

```
    (*(unsigned long*)(0x40021C00) & (~0x00008000)) | (0x00004000);

    // PI10
    *(unsigned long*)(0x40022000) =
    (*(unsigned long*)(0x40022000) & (~0x00200000)) | (0x00100000);

    while(1) {

        // 11000000 -> PG6, PG7
        *(unsigned long*)(0x40021814) |= 0x40;
        *(unsigned long*)(0x40021814) |= 0x80;
        for(j=0; j<2000000; j++){}
        *(unsigned long*)(0x40021814) &= ~0x40;
        *(unsigned long*)(0x40021814) &= ~0x80;
        for(j=0; j<2000000; j++){}

        // 00110000 -> PG8, PH2
        *(unsigned long*)(0x40021814) |= 0x100;
        *(unsigned long*)(0x40021C14) |= 0x4;
        for(j=0; j<2000000; j++){}
        *(unsigned long*)(0x40021814) &= ~0x100;
        *(unsigned long*)(0x40021C14) &= ~0x4;
        for(j=0; j<2000000; j++){}

        // 00001100 -> PH3, PH6
        *(unsigned long*)(0x40021C14) |= 0x8;
        *(unsigned long*)(0x40021C14) |= 0x40;
        for(j=0; j<2000000; j++){}
        *(unsigned long*)(0x40021C14) &= ~0x8;
        *(unsigned long*)(0x40021C14) &= ~0x40;
        for(j=0; j<2000000; j++){}

        // 00000011 -> PH7, PI10
        *(unsigned long*)(0x40021C14) |= 0x80;
        *(unsigned long*)(0x40022014) |= 0x400;
        for(j=0; j<2000000; j++){}
        *(unsigned long*)(0x40021C14) &= ~0x80;
        *(unsigned long*)(0x40022014) &= ~0x400;
        for(j=0; j<2000000; j++){}
    }
}
```

5 Выводы

Поставленная цель лабораторной работы была успешно достигнута. Разработана программа управления светодиодами на оценочной плате с использованием прямого доступа к регистрам микроконтроллера STM32F200. Реализовано последовательное включение светодиодов попарно в соответствии с заданными шаблонами.

В ходе практической части работы был реализован дополнительный режим управления светодиодами с использованием группового смещения битов, который продемонстрировал корректную и стабильную работу.

В процессе выполнения лабораторной работы были изучены основные принципы работы с портами ввода-вывода микроконтроллера, а также методы прямого обращения к памяти через регистры. Были освоены побитовые операции, позволяющие управлять отдельными выводами без влияния на остальные.

Кроме того, были получены практические навыки работы в среде разработки Keil μVision5, включая создание проекта, настройку микроконтроллера и написание программ на языке C.

Таким образом, цель лабораторной работы достигнута, а полученные знания могут быть использованы при дальнейшем программировании микроконтроллеров.